CENTRO UNIVERSITARIO DE CIENCIAS EXACTAS E INGENIERÍAS

División de Tecnologías para la Integración Ciber-Humana

Departamento de Innovación Basada en la Información y el Conocimiento

Práctica 2: ADALINE

Clasificación binaria del estado de peso a partir de altura y peso

Material complementario, fundamentos teóricos y descripción de la práctica

Asignatura	Sistemas Inteligentes 3 – Redes Neuronales Artificiales
Tema	ADALINE, regla delta/LMS y clasificación lineal
Producto	Código fuente + reporte técnico

Idea central de la práctica

Implementar desde cero una neurona ADALINE, entrenarla con datos reales de altura y peso, estudiar la convergencia del algoritmo y evaluar qué tan bien una frontera lineal aproxima una clasificación definida mediante el índice de masa corporal.

1. Propósito y resultados de aprendizaje

Esta práctica introduce ADALINE (*Adaptive Linear Neuron*) como un modelo lineal entrenado mediante minimización de error cuadrático. El énfasis no está únicamente en obtener una clasificación, sino en comprender cómo una función de costo conduce a una regla de aprendizaje basada en gradiente. Este paso conecta los modelos lineales estudiados al inicio del curso con los métodos de entrenamiento utilizados posteriormente en redes multicapa [1, 2, 3, 4].

Al finalizar la práctica, el alumno deberá ser capaz de:

- distinguir el criterio de aprendizaje de ADALINE respecto al perceptrón;
- derivar e implementar la regla delta o regla de Widrow–Hoff;
- analizar el efecto de la razón de aprendizaje η sobre la convergencia;
- trabajar con datos reales y realizar un preprocesamiento reproducible;
- separar adecuadamente datos de entrenamiento y prueba para evaluar generalización;
- interpretar la frontera de decisión y reconocer las limitaciones de un clasificador lineal.

Importante

El IMC se utilizará exclusivamente para construir la etiqueta deseada de cada patrón. No debe utilizarse como entrada de la red, porque eso introduciría fuga de información y convertiría la clasificación en un problema trivial.

2. Fundamentos de ADALINE

2.1. Modelo

ADALINE fue desarrollado en el contexto de los trabajos de Widrow y Hoff. A diferencia del perceptrón, durante el entrenamiento utiliza directamente la respuesta lineal de la unidad. Aggarwal señala que el aprendizaje de Widrow–Hoff puede interpretarse como clasificación por mínimos cuadrados y que también se conoce como ADALINE, regla delta o LMS [3]. Goodfellow, Bengio y Courville lo sitúan entre los precursores de los modelos lineales modernos y señalan que su adaptación de pesos constituye un caso temprano de descenso de gradiente estocástico [4].

La salida lineal se expresa como

$$y(k) = \mathbf{W}^T(k)\mathbf{X}(k) + b(k). \tag{1}$$

Para esta práctica, el vector de entrada contiene dos variables:

$$\mathbf{X}(k) = \begin{bmatrix} x_1(k) \\ x_2(k) \end{bmatrix} = \begin{bmatrix} \text{altura}(k) \\ \text{peso}(k) \end{bmatrix}. \tag{2}$$

Notación. El índice k representa la actualización del algoritmo. $\mathbf{X}(k)$ es el patrón presentado en la actualización k; $\mathbf{W}(k)$ es el vector de pesos antes de la actualización; $b(k)$ es el *bias*; $y(k)$ es la salida lineal; $d(k)$ es la salida deseada y

$$e(k) = d(k) - y(k) \tag{3}$$

es el error. Esta convención sigue la forma iterativa habitual en programación y teoría de control: *estado/parámetro nuevo = estado/parámetro actual + corrección*.

El valor $y(k)$ es continuo. Para convertirlo en una decisión binaria después del entrenamiento puede utilizarse

$$\hat{d}(k) = \begin{cases} +1, & y(k) \geq 0, \\ -1, & y(k) < 0. \end{cases} \tag{4}$$

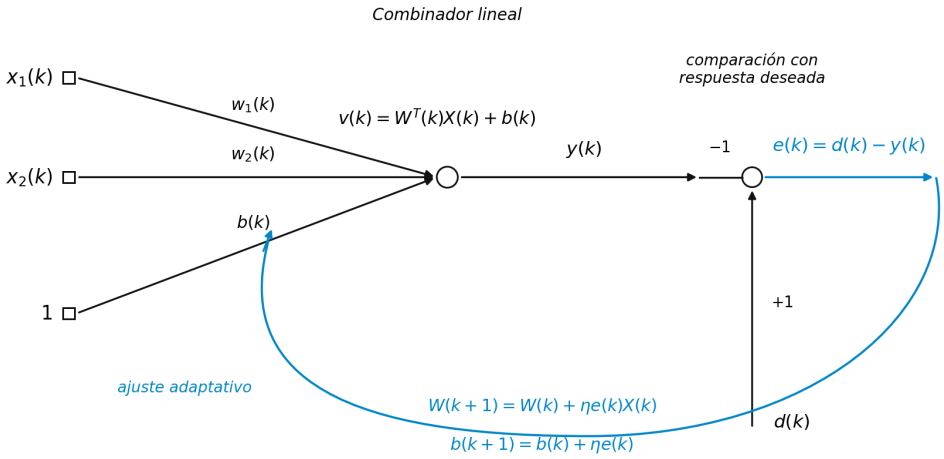


Figura 1: Estructura de entrenamiento ADALINE/LMS basada en el lazo de filtrado adaptativo de Haykin [1]. La respuesta deseada $d(k)$ se compara con $y(k)$; el error $e(k)$ retroalimenta la actualización de $\mathbf{W}(k)$ y $b(k)$.

Diferencia clave con el perceptrón

ADALINE calcula el error de entrenamiento con la salida lineal $y(k)$. La función signo se reserva para la decisión de clase. En el perceptrón, en cambio, el error está ligado a la salida discretizada. Esta diferencia permite formular el aprendizaje de ADALINE como minimización de una función de error cuadrático [3].

2.2. Comparación con el perceptrón

Aspecto	Perceptrón	ADALINE
Salida utilizada para entrenar	Salida umbralizada / decisión de clase	Salida lineal $y(k)$
Criterio de aprendizaje	Corrección asociada a clasificación	Minimización de error cuadrático
Regla	Regla del perceptrón	Regla delta / Widrow–Hoff / LMS
Tipo de frontera	Lineal	Lineal
Papel del gradiente	No se plantea necesariamente como mínimos cuadrados	Se deriva explícitamente de una función de costo
Convergencia práctica	Depende de separabilidad para clasificación perfecta	Depende de la superficie cuadrática, el escalamiento y η; puede obtener una solución de mínimos cuadrados aun con errores de clasificación

2.3. Función de costo y obtención de la regla delta

Sea $d(k)$ la salida deseada asociada al patrón $\mathbf{X}(k)$, y sea $y(k)$ la salida lineal calculada con los parámetros actuales. Se define el error instantáneo como

$$e(k) = d(k) - y(k). \tag{5}$$

Una función de costo conveniente es el error cuadrático:

$$E(k) = \frac{1}{2}e^2(k) = \frac{1}{2}\left[d(k) - y(k)\right]^2. \tag{6}$$

El objetivo es modificar cada peso $w_i(k)$ en la dirección que disminuya $E(k)$. Aplicando descenso de gradiente en forma iterativa:

$$w_i(k+1) = w_i(k) - \eta \frac{\partial E(k)}{\partial w_i(k)}. \tag{7}$$

Como

$$y(k) = \sum_i w_i(k)x_i(k) + b(k), \tag{8}$$

por regla de la cadena:

$$\frac{\partial E(k)}{\partial w_i(k)} = \frac{\partial E}{\partial e}\frac{\partial e}{\partial y}\frac{\partial y}{\partial w_i} \tag{9}$$

$$= e(k)(-1)x_i(k) \tag{10}$$

$$= -e(k)x_i(k). \tag{11}$$

Sustituyendo en (7):

$$w_i(k+1) = w_i(k) + \eta\, e(k)x_i(k), \tag{12}$$

$$b(k+1) = b(k) + \eta\, e(k). \tag{13}$$

En forma vectorial, que será la notación principal utilizada en la práctica:

$$\boxed{\mathbf{W}(k+1) = \mathbf{W}(k) + \eta\, e(k)\mathbf{X}(k)}. \tag{14}$$

Mehlig deriva esta regla para unidades lineales a partir de una energía cuadrática y destaca que su ventaja es precisamente contar con una función que puede analizarse mediante gradiente [2]. Aggarwal presenta la misma estructura de actualización en el contexto de mínimos cuadrados y Widrow–Hoff [3].

2.4. Modalidades de descenso de gradiente: estocástico, por lote y mini-lote

La regla delta expresada como $\mathbf{W}(k+1) = \mathbf{W}(k) +$ corrección puede aplicarse con distintas frecuencias de actualización. La diferencia no está en el modelo ADALINE, sino en cuántos patrones se utilizan para estimar el gradiente antes de modificar los pesos. Las tres modalidades más comunes son descenso de gradiente por lote (*batch*), descenso de gradiente estocástico (SGD) y descenso por mini-lotes (*mini-batch*).

2.4.1. Descenso de gradiente por lote (Batch Gradient Descent)

En entrenamiento por lote se utilizan los N patrones de entrenamiento para calcular el gradiente de la función de costo antes de realizar una sola actualización. Aquí k identifica el número de actualización de los parámetros. Si

$$J\big[\mathbf{W}(k), b(k)\big] = \frac{1}{2N}\sum_{i=1}^{N} e_i^2(k), \tag{15}$$

entonces la actualización puede escribirse como

$$\mathbf{W}(k+1) = \mathbf{W}(k) + \eta\frac{1}{N}\sum_{i=1}^{N} e_i(k)\mathbf{X}_i, \tag{16}$$

$$b(k+1) = b(k) + \eta\frac{1}{N}\sum_{i=1}^{N} e_i(k). \tag{17}$$

Una época completa produce una sola actualización. La dirección utilizada corresponde al gradiente exacto del error empírico del conjunto de entrenamiento. Su comportamiento suele ser suave y determinista si se conservan el mismo conjunto e inicialización, pero cada actualización requiere procesar todos los patrones.

2.4.2. Descenso de gradiente estocástico (Stochastic Gradient Descent, SGD)

En la modalidad estocástica se presenta un patrón $\mathbf{X}(k)$ y se actualizan inmediatamente los parámetros. En este caso k identifica simultáneamente la actualización y el patrón presentado en ese instante:

$$y(k) = \mathbf{W}^T(k)\mathbf{X}(k) + b(k), \tag{18}$$

$$e(k) = d(k) - y(k), \tag{19}$$

$$\mathbf{W}(k+1) = \mathbf{W}(k) + \eta\, e(k)\mathbf{X}(k), \tag{20}$$

$$b(k+1) = b(k) + \eta\, e(k). \tag{21}$$

En ADALINE, esta forma patrón a patrón está estrechamente relacionada con el algoritmo LMS de Widrow–Hoff. Cada actualización utiliza una estimación del gradiente basada en una sola muestra; por ello la trayectoria puede fluctuar alrededor de la dirección de descenso. A cambio, se realizan muchas actualizaciones por época, se requiere poca memoria y puede observarse progreso desde los primeros patrones. Es recomendable mezclar (*shuffle*) el orden de los datos al inicio de cada época para evitar efectos sistemáticos debidos al orden de presentación.

2.4.3. Descenso de gradiente por mini-lotes (Mini-batch Gradient Descent)

El entrenamiento por mini-lotes utiliza subconjuntos de M patrones, con $1 < M < N$. Para un mini-lote $\mathcal{B}_k$ de tamaño M:

$$\mathbf{W}(k+1) = \mathbf{W}(k) + \eta\frac{1}{M}\sum_{i\in\mathcal{B}_k} e_i(k)\mathbf{X}_i, \tag{22}$$

$$b(k+1) = b(k) + \eta\frac{1}{M}\sum_{i\in\mathcal{B}_k} e_i(k). \tag{23}$$

Después de actualizar los parámetros se procesa el siguiente mini-lote. Esta modalidad representa un punto intermedio: reduce la variabilidad respecto a SGD y actualiza con mayor frecuencia que batch. Además, en implementaciones de mayor escala permite aprovechar operaciones vectorizadas. Los casos extremos son útiles para interpretar el algoritmo: $M = 1$ equivale a SGD y $M = N$ equivale a batch.

2.4.4. Comparación general

Modalidad	Patrones por actualización	Actualizaciones por época	Comportamiento típico
Estocástico (SGD)	1	N	Actualizaciones frecuentes; gradiente variable; bajo uso de memoria.
Mini-lote	M, con $1 < M < N$	$\approx N/M$	Compromiso entre estabilidad y frecuencia de actualización.
Lote (Batch)	N	1	Gradiente exacto del conjunto; trayectoria más suave; mayor costo por actualización.

Una época se define como una pasada completa por todos los patrones de entrenamiento. Por ello, una época contiene una actualización en batch, aproximadamente N/M actualizaciones en mini-batch y N actualizaciones en SGD. Al comparar modalidades deben mantenerse, en lo posible, el mismo conjunto de entrenamiento, la misma inicialización y el mismo criterio de evaluación.

3. Razón de aprendizaje η y convergencia

La razón de aprendizaje η determina la magnitud del cambio aplicado a los pesos. Un valor demasiado pequeño produce aprendizaje lento; uno excesivamente grande puede provocar oscilaciones o divergencia. La escala apropiada depende de la geometría de la función de costo y, por tanto, de la escala y correlación de los datos. Goodfellow, Bengio y Courville muestran que, para funciones aproximadamente cuadráticas, los valores propios del Hessiano determinan la escala razonable del paso de gradiente [4].

Valor de η	Comportamiento esperado
Muy pequeño	Cambios pequeños; aprendizaje estable pero lento.
Adecuado	Disminución sostenida del error y convergencia práctica.
Grande	Puede atravesar repetidamente la región del mínimo y oscilar.
Excesivo	El error puede crecer y el algoritmo puede divergir.

3.1. Análisis matemático complementario

Para la versión batch de ADALINE, con N patrones y una matriz X que contiene las entradas, puede escribirse

$$J\big[\mathbf{W}(k)\big] = \frac{1}{2N}\,\|\mathbf{d} - X\mathbf{W}(k)\|_2^2\,, \tag{24}$$

$$\nabla J\big[\mathbf{W}(k)\big] = -\frac{1}{N}X^T\,[\mathbf{d} - X\mathbf{W}(k)]\,, \tag{25}$$

$$H = \frac{1}{N}X^T X. \tag{26}$$

Al tratarse de una función cuadrática, una condición estándar de convergencia para descenso de gradiente con paso fijo es

$$\boxed{0 < \eta < \frac{2}{\lambda_{\text{máx}}(H)}}. \tag{27}$$

Esta expresión debe interpretarse como guía para el caso batch. En SGD y mini-batch, el gradiente utilizado en cada actualización es una estimación del gradiente global, por lo que el MSE no necesariamente disminuye de manera monótona después de cada actualización. El comportamiento práctico depende también del orden de presentación, del escalamiento, del tamaño del mini-lote y de la distribución de los datos.

Escalamiento de las entradas

Altura y peso se expresan en escalas distintas. Estandarizar las variables suele mejorar el condicionamiento del problema y facilita la selección de η. Si se estandariza, la media y desviación estándar deben calcularse únicamente con el conjunto de entrenamiento y después reutilizarse sin cambios sobre el conjunto de prueba.

Un valor de η que funciona adecuadamente en batch no tiene por qué producir exactamente el mismo comportamiento en SGD o mini-batch. Con actualizaciones más frecuentes, η debe evaluarse experimentalmente observando la curva de error y la estabilidad del entrenamiento. Para SGD es común mezclar los patrones en cada época; para mini-batch, se recomienda formar los lotes después de mezclar el conjunto de entrenamiento.

4. Descripción de la práctica

El alumno deberá localizar una base de datos pública que incluya, al menos, peso y altura de personas pertenecientes a una población identificable (por ejemplo, localidad, ciudad, estado, región o país). A partir de estos datos deberá construir un problema de clasificación binaria y resolverlo con una ADALINE implementada desde cero.

Nota sobre la modalidad de gradiente

Para esta actividad, el alumno puede elegir libremente cualquiera de las tres modalidades de entrenamiento estudiadas: **por lote (batch), por mini-lotes (mini-batch) o estocástica (SGD)**. No es obligatorio implementar las tres. La modalidad elegida debe programarse desde cero, identificarse claramente en el reporte y documentarse con sus parámetros correspondientes (por ejemplo, el tamaño M si se emplea mini-batch).

4.1. Selección y documentación de la base de datos

- La base debe ser real, pública o de uso académico permitido y tener una fuente verificable.
- Debe contener altura y peso, o variables que puedan convertirse claramente a metros y kilogramos.
- El alumno debe registrar: nombre de la base, institución o autor, población representada, número de registros, variables utilizadas y enlace o referencia de origen.
- Debe documentar registros eliminados, valores faltantes, conversiones de unidades y cualquier filtrado realizado.
- No deben utilizarse datos personales identificables que no sean necesarios para la práctica.

No se acepta

Generar artificialmente una base de datos de altura y peso y presentarla como datos reales. Tampoco se permite utilizar el IMC como variable de entrada de la red.

4.2. Cálculo del IMC y construcción de las clases

Para cada registro deberá calcularse el índice de masa corporal:

$$\text{IMC} = \frac{\text{peso (kg)}}{[\text{altura (m)}]^2}. \tag{28}$$

Categoría	Intervalo de IMC
Bajo peso	$\text{IMC} < 18.5$
Peso normal	$18.5 \leq \text{IMC} < 25$
Sobrepeso	$25 \leq \text{IMC} < 30$
Obesidad	$\text{IMC} \geq 30$

Para esta práctica las cuatro categorías se agruparán en dos clases:

$$C_1 = \{\text{Bajo peso, Peso normal}\} \quad \Longrightarrow \quad d = -1, \tag{29}$$

$$C_2 = \{\text{Sobrepeso, Obesidad}\} \quad \Longrightarrow \quad d = +1. \tag{30}$$

El punto de separación se encuentra en IMC $= 25$. Si las entradas son altura h y peso m, la frontera ideal queda definida por

$$\boxed{m = 25h^2}. \tag{31}$$

Por tanto, el problema no es estrictamente lineal en el plano altura–peso. En un intervalo acotado de alturas, la curvatura puede ser moderada y una recta puede aproximarla. Esta característica es intencional: permitirá observar qué tan lejos puede llegar una unidad lineal y qué errores aparecen cerca de la frontera.

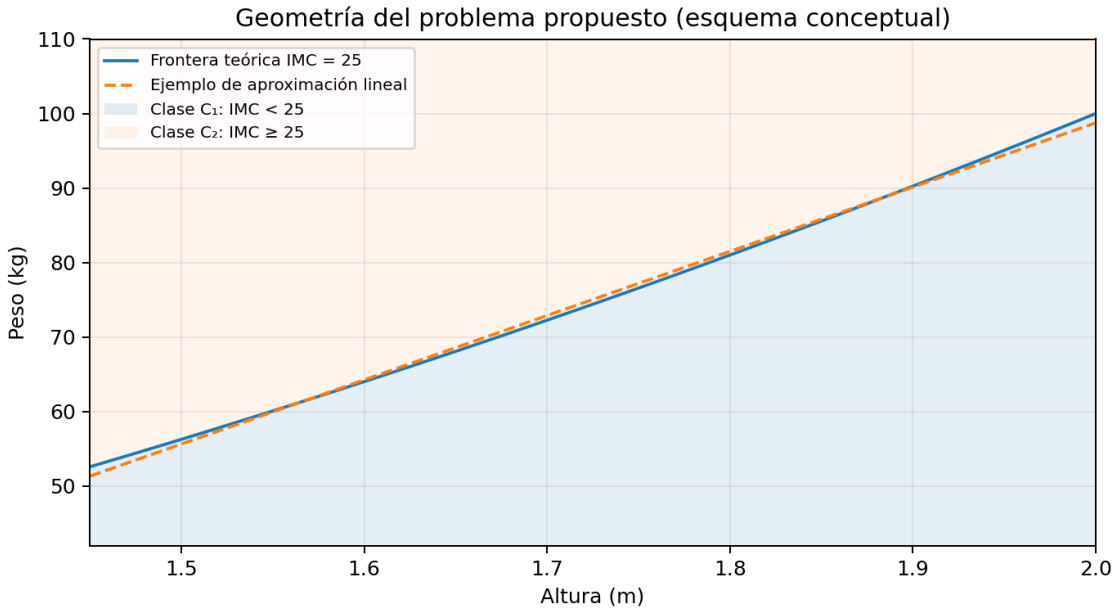


Figura 2: Frontera teórica IMC $= 25$ y ejemplo de aproximación lineal. La figura es conceptual y no representa una base de datos real.

4.3. Normalización y escalamiento de los datos

La magnitud de las variables de entrada afecta directamente al entrenamiento de ADALINE. En la actualización

$$\mathbf{W}(k+1) = \mathbf{W}(k) + \eta\, e(k)\mathbf{X}(k), \tag{32}$$

una variable con valores numéricos mayores puede producir cambios de mayor magnitud en los pesos y dominar el gradiente. En este problema, por ejemplo, la altura expresada en metros suele estar alrededor de 1.4–2.0, mientras que el peso puede tomar valores de varias decenas o centenas de kilogramos. Por esta razón conviene escalar cada característica antes del entrenamiento.

El escalamiento debe aplicarse únicamente a las entradas de la red. El IMC y las etiquetas de clase deben calcularse primero con la altura y el peso en sus unidades físicas originales. Después de construir las etiquetas y separar los datos en entrenamiento y prueba, se calculan los parámetros del método de escalamiento usando solamente el conjunto de entrenamiento.

4.3.1. Normalización Min–Max

La normalización Min–Max transforma cada característica a un intervalo definido. La forma más común lleva los datos al intervalo $[0, 1]$. Para una característica x_j y el patrón k:

$$\bar{x}_j(k) = \frac{x_j(k) - x_{j,\text{mín}}}{x_{j,\text{máx}} - x_{j,\text{mín}}}. \tag{33}$$

Aquí $x_{j,\text{mín}}$ y $x_{j,\text{máx}}$ son el mínimo y máximo de la característica j calculados exclusivamente con los datos de entrenamiento. Después, los mismos valores se utilizan para transformar los datos de prueba.

Si se desea el intervalo $[-1, 1]$, puede utilizarse

$$\bar{x}_j(k) = 2\frac{x_j(k) - x_{j,\text{mín}}}{x_{j,\text{máx}} - x_{j,\text{mín}}} - 1. \tag{34}$$

Min–Max conserva la relación relativa entre los valores y produce un rango acotado, lo cual facilita la selección de η. Sin embargo, es sensible a valores atípicos: un mínimo o máximo extremo puede comprimir gran parte de los datos dentro de una región pequeña del intervalo.

4.3.2. Estandarización o Standard Scaling

La estandarización centra cada característica alrededor de cero y la escala usando su desviación estándar. Para una característica x_j:

$$z_j(k) = \frac{x_j(k) - \mu_j}{\sigma_j}, \tag{35}$$

con

$$\mu_j = \frac{1}{N}\sum_{k=1}^{N} x_j(k), \tag{36}$$

$$\sigma_j = \sqrt{\frac{1}{N}\sum_{k=1}^{N} [x_j(k) - \mu_j]^2}. \tag{37}$$

Después de la transformación, cada variable del conjunto de entrenamiento tendrá aproximadamente media cero y desviación estándar unitaria. Los valores μ_j y σ_j obtenidos durante el entrenamiento deben conservarse y utilizarse sin modificarlos para transformar los datos de prueba o cualquier patrón nuevo.

Standard Scaling suele ser especialmente conveniente para algoritmos entrenados mediante gradiente, porque coloca características con unidades y magnitudes distintas en escalas comparables. Esto puede mejorar el acondicionamiento del problema y hacer que una misma razón de aprendizaje η produzca actualizaciones más balanceadas entre los pesos.

4.3.3. Consideraciones prácticas

- El conjunto debe dividirse en entrenamiento y prueba antes de calcular mínimos, máximos, medias o desviaciones estándar. Calcular estos parámetros con toda la base introduce fuga de información (*data leakage*).
- Min–Max y Standard Scaling se calculan por característica; no debe mezclarse la altura y el peso para obtener un único mínimo, máximo, media o desviación estándar.
- Los parámetros obtenidos con entrenamiento deben reutilizarse exactamente en prueba y generalización. No se vuelven a ajustar con los datos nuevos.
- Con Min–Max, un patrón de prueba puede quedar fuera de $[0, 1]$ si contiene valores menores o mayores que los observados durante el entrenamiento. Esto no implica necesariamente un error y no debe recortarse automáticamente sin justificación.
- Si una característica tiene desviación estándar cero o $x_{j,\text{máx}} = x_{j,\text{mín}}$, no aporta variación y la división correspondiente debe tratarse explícitamente para evitar una operación indefinida.
- Ambos métodos pueden verse afectados por valores atípicos. Antes de escalar, el alumno deberá inspeccionar la base y justificar cualquier tratamiento de valores extremos.

Para esta práctica el alumno deberá implementar al menos Min–Max y Standard Scaling con operaciones básicas, comparar brevemente su efecto sobre el entrenamiento y seleccionar uno para el modelo final. Debe reportar los parámetros obtenidos para altura y peso, así como justificar el método elegido.

4.4. Diseño de la red

La arquitectura mínima deberá tener dos entradas y una salida lineal. Durante el entrenamiento se utilizará la notación discreta en k:

$$\mathbf{X}(k) = \begin{bmatrix} \text{altura}(k) \\ \text{peso}(k) \end{bmatrix}, \tag{38}$$

$$y(k) = \mathbf{W}^T(k)\mathbf{X}(k) + b(k). \tag{39}$$

El alumno deberá:

- definir claramente la representación del bias;
- especificar cómo se inicializan los pesos;
- elegir y justificar η;
- definir el número máximo de épocas y el criterio de paro;
- indicar el método de escalamiento utilizado (Min–Max o Standard Scaling), sus parámetros y la justificación de la elección;
- utilizar la función signo únicamente para convertir la salida lineal en una etiqueta de clase.

4.5. Implementación obligatoria desde cero

El alumno debe programar el algoritmo de ADALINE. No se permite utilizar una implementación ya existente de redes neuronales o de ADALINE como sustituto del algoritmo desarrollado por el alumno.

Permitido	**No permitido**
Lectura de archivos, manejo de arreglos, generación de gráficas y operaciones matemáticas básicas.	Clases o funciones que implementen ADALINE, perceptrón, redes neuronales o entrenamiento automático del modelo.
Programar manualmente producto punto, error, MSE, actualización de $\mathbf{W}(k)$ y $b(k)$ y clasificación.	Utilizar regresión lineal, clasificadores u optimizadores de una biblioteca para obtener directamente los pesos finales.
Usar bibliotecas para métricas solo si el alumno también muestra cómo se calculan.	Usar una red de una biblioteca y limitarse a cambiar parámetros.

4.6. Algoritmo mínimo a implementar

1. Leer y limpiar la base de datos.
2. Convertir altura y peso a unidades consistentes.
3. Calcular el IMC y construir la etiqueta bipolar $d \in \{-1, +1\}$.
4. Separar los datos en entrenamiento y prueba antes de cualquier ajuste de escalamiento.
5. Calcular con el conjunto de entrenamiento los parámetros de Min–Max y Standard Scaling para cada característica, comparar ambos métodos y seleccionar el escalamiento del modelo final.
6. Transformar entrenamiento y prueba con los parámetros obtenidos exclusivamente del conjunto de entrenamiento; no recalcularlos con los datos de prueba.
7. Inicializar $\mathbf{W}(0)$ y $b(0)$.
8. Para cada época, recorrer todos los patrones de entrenamiento de acuerdo con la modalidad elegida: patrón individual, mini-lotes o lote completo.
9. Calcular las salidas lineales y los errores de los patrones que correspondan a la actualización actual.
10. Actualizar $\mathbf{W}(k)$ y $b(k)$ con la regla delta según la modalidad elegida, obteniendo $\mathbf{W}(k+1)$ y $b(k+1)$: inmediatamente para SGD, después de cada mini-lote o una vez al final del lote completo.
11. Al final de cada época calcular y almacenar el MSE.
12. Detener al alcanzar el criterio establecido o el máximo de épocas.
13. Evaluar el modelo final con el conjunto de prueba, sin modificar nuevamente los pesos.

4.7. Modalidad de entrenamiento que deberá documentarse

El alumno deberá indicar explícitamente si implementó entrenamiento estocástico, por lote o por mini-lotes. En caso de mini-lotes deberá indicar el tamaño M y explicar cómo se forman los grupos. También deberá señalar si los datos se mezclan al inicio de cada época.

Como actividad complementaria de análisis, se recomienda comparar al menos dos modalidades utilizando la misma partición de datos y una inicialización equivalente. El objetivo no es únicamente decidir cuál es más rápida, sino observar cómo cambian la suavidad de la curva de error, la velocidad de reducción del MSE y la sensibilidad a η.

5. Resultados obligatorios

5.1. Curva de aprendizaje

Al finalizar cada época deberá calcularse una medida global del error. Se recomienda

$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}[d_i - y_i]^2\,. \tag{40}$$

Se deberá graficar MSE contra época y explicar si la curva muestra convergencia, oscilación, estancamiento o divergencia.

5.2. Estudio experimental de η y de la modalidad de actualización

El alumno deberá entrenar la red con al menos tres valores de η que permitan comparar velocidad y estabilidad. No se exige usar valores específicos: deben escogerse considerando el escalamiento de los datos y la modalidad de actualización utilizada. Si se comparan SGD, mini-batch y batch, debe indicarse que el mismo valor de η puede producir comportamientos distintos porque cambia la frecuencia y la variabilidad de las actualizaciones.

5.3. Frontera o superficie de decisión

En dos dimensiones, la frontera de decisión de ADALINE es una recta:

$$w_{1,f}x_1 + w_{2,f}x_2 + b_f = 0. \tag{41}$$

Si $w_{2,f} \neq 0$, puede despejarse

$$x_2 = -\frac{w_{1,f}x_1 + b_f}{w_{2,f}}. \tag{42}$$

La gráfica deberá mostrar los patrones de ambas clases, la frontera aprendida y, de manera recomendada, la curva teórica IMC $= 25$. Si las entradas fueron estandarizadas, la figura final debe interpretarse o reconstruirse en las unidades originales de altura y peso.

5.4. Pruebas de generalización

Los datos de prueba deben mantenerse fuera del entrenamiento. Una división 70/30 u 80/20 es adecuada si el tamaño de la base lo permite; el alumno deberá indicar la proporción utilizada. Es recomendable preservar la proporción de ambas clases en la separación.

La exactitud puede calcularse como

$$\text{Accuracy} = \frac{\text{clasificaciones correctas}}{\text{total de patrones de prueba}}. \tag{43}$$

Además del accuracy, deberá construirse una matriz de confusión. Para un problema binario puede utilizarse el siguiente formato:

Clase real / predicha	C_1	C_2
C_1	Verdaderos C_1	C_1 clasificados como C_2
C_2	C_2 clasificados como C_1	Verdaderos C_2

Prueba útil

Identificar ejemplos de prueba cercanos a IMC $= 25$ y analizar si la mayoría de los errores se concentra alrededor de la frontera. Esto permite relacionar los errores con la aproximación lineal realizada por ADALINE.

6. Entregables y contenido mínimo del reporte

Elemento	**Contenido esperado**
1. Base de datos	Fuente, población, tamaño, variables y proceso de limpieza.
2. Formulación	Cálculo de IMC, definición de C_1/C_2 y codificación de etiquetas.
3. Diseño de ADALINE	Arquitectura, entradas, salida, bias, inicialización, modalidad de entrenamiento y criterio de paro.
4. Algoritmo	Descripción y/o pseudocódigo; el código debe implementar la regla delta y la estrategia de actualización seleccionada.
5. Razón de aprendizaje	Comparación de al menos tres valores de η y justificación del seleccionado; documentar tamaño de mini-lote si aplica.
6. Curva de error	MSE contra época, con interpretación.
7. Frontera de decisión	Datos, frontera ADALINE y análisis respecto a IMC $= 25$.
8. Generalización	Resultados de prueba, accuracy y matriz de confusión.

Elemento	Contenido esperado
9. Discusión	Errores, limitaciones del modelo, efecto del escalamiento, modalidad de gradiente y relación con linealidad.
10. Conclusiones	Qué aprendió el modelo, qué no puede representar y cómo mejoraría con una red multicapa.

Archivos: entregar el código fuente por separado y un reporte técnico en PDF. El código debe ser reproducible y permitir ejecutar nuevamente el entrenamiento con la base utilizada.

7. Preguntas orientadoras para la discusión

1. ¿Por qué ADALINE calcula el error con la salida lineal y no con la salida de la función signo?
2. ¿Qué efecto tuvo η sobre la velocidad de convergencia y sobre la estabilidad del entrenamiento?
3. Compare Min–Max y Standard Scaling. ¿Qué efecto observó sobre la convergencia, la magnitud del gradiente y la sensibilidad a η?
4. ¿Los errores de clasificación se concentran cerca de IMC $= 25$? Explique por qué.
5. ¿La frontera aprendida coincide con la frontera teórica $m = 25h^2$? ¿Por qué no debería esperarse una coincidencia exacta?
6. ¿Qué limitación del modelo lineal queda en evidencia en este problema?
7. ¿Por qué una red multicapa con activaciones no lineales es un siguiente paso natural para separar las cuatro categorías originales?
8. Compare conceptualmente SGD, mini-batch y batch. ¿Cómo cambia el número de actualizaciones por época y qué efecto observó o esperaría observar en la curva de error?

8. Relación con la siguiente práctica

En esta práctica se reducen las cuatro categorías de IMC a dos clases para estudiar una única unidad lineal. En la práctica posterior se retomará el mismo problema conservando las cuatro categorías: bajo peso, peso normal, sobrepeso y obesidad. Esto permitirá introducir una arquitectura multicapa y estudiar cómo las transformaciones no lineales y múltiples regiones de decisión superan las limitaciones de una sola frontera lineal.

Idea de continuidad

Perceptrón $\longrightarrow$ ADALINE $\longrightarrow$ Red multicapa. El cambio esencial es pasar de una regla de corrección para clasificación a una función de costo diferenciable y, después, extender el mismo principio de gradiente a varias capas.

9. Fuentes de referencia y lecturas recomendadas

Las siguientes fuentes sustentan el material teórico utilizado en esta guía. Para esta práctica se recomienda especialmente consultar Haykin, Mehlig y Aggarwal.

Referencias

[1] S. Haykin, *Neural Networks and Learning Machines*, 3rd ed. Upper Saddle River, NJ, USA: Pearson/Prentice Hall, 2009. Véanse especialmente el Capítulo 3, "The Least-Mean-Square Algorithm", y la discusión de aprendizaje batch y on-line del Capítulo 4.

[2] B. Mehlig, *Machine Learning with Neural Networks*, 2021. arXiv:1901.05639v4 [cs.LG]. Véanse la Sección 5.3, "Gradient descent for linear units", y el Capítulo 6, especialmente las secciones 6.2–6.5 sobre stochastic gradient descent, preprocesamiento de datos de entrada y adaptación de la razón de aprendizaje.

[3] C. C. Aggarwal, *Neural Networks and Deep Learning: A Textbook*. Cham, Switzerland: Springer, 2018. Véanse las Secciones 2.2.2 y 2.2.2.1, "Least-Squares Regression" y "Widrow-Hoff Learning", la Sección 3.2.8 sobre mini-batch stochastic gradient descent y la Sección 3.3.2 sobre feature preprocessing. DOI: 10.1007/978-3-319-94463-0.

[4] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016. Véanse el Capítulo 1 para contexto histórico de ADALINE, la Sección 5.9 sobre stochastic gradient descent y el Capítulo 8 para algoritmos de optimización y entrenamiento por mini-lotes.

[5] P. Isasi Viñuela and I. M. Galván León, *Redes de Neuronas Artificiales: Un Enfoque Práctico*. Madrid, España: Pearson Educación, 2004. Véanse la introducción histórica y los capítulos iniciales sobre modelos neuronales y aprendizaje.

Nota sobre la condición de convergencia

La condición $0 < \eta < 2/\lambda_{\text{máx}}(H)$ presentada en la Sección 3.1 corresponde al análisis estándar de descenso de gradiente con paso fijo para una función cuadrática batch. En entrenamiento estocástico o por mini-lotes, la estabilidad y la convergencia dependen además de la variabilidad de las estimaciones del gradiente, del orden de presentación y del tamaño del lote.